

02 需求工程复习讲义

目录

2. 需求工程	2
2.1 需求工程的含义	2
2.2 问题域	2
2.3 需求层次	3
业务需求	3
用户需求	3
系统级需求	4
2.4 需求类型	4
功能需求	4
非功能需求	4
外部接口需求	5
约束	5
数据需求	6
2.5 AI 辅助需求获取	6
2.6 用例建模	7
基本元素	7
include 与 extend	7
用例粒度	8
Cockburn 层次	8
2.7 用例描述模板	8

目录	2
2.8 用户故事与 BDD	9
2.9 需求分析模型	10
2.10 概念类图	10
从文本提取概念类	11
关联与多重性	12
2.11 顺序图	12
2.12 状态图	13
2.13 OCL	14
2.14 SRS 与需求验证	15
2.15 好需求的标准	15
2.16 需求追踪	16
2.17 歧义与冲突	17
2.18 Spec 驱动开发	18
对应考试模板	18
7.4 需求题答题模板	18
写业务需求	18
写用户需求	18
写系统级功能需求	19
写非功能需求	19
7.5 概念类图题模板	19
7.6 顺序图题模板	20
7.7 状态图题模板	21
快速背诵清单	21
需求工程	21

范围：02-01-需求工程基础.pdf 到 02-05-需求文档化验证与冲突检测.pdf；结合 Exam/Review 中需求类型、用例、概念类图、顺序图和状态图内容。结构：先整理 PPT 主线，再放对应考试模板和快速背诵清单。

2. 需求工程

2.1 需求工程的含义

需求工程覆盖所有围绕需求展开的活动。

核心活动：

- 需求获取
- 需求分析
- 需求规格说明
- 需求验证
- 需求管理

需求工程不是简单“问用户要功能”。它需要理解问题域、利益相关者、业务流程、约束条件、数据规则和外部系统。

2.2 问题域

问题域包含：

- 业务目标
- 业务流程
- 领域术语
- 组织规则
- 相关人员
- 现有系统
- 数据对象

- 外部约束

需求工程的关键是先理解问题域，再定义系统边界和解决方案。

2.3 需求层次

业务需求

回答“为什么建设系统”。

关注：

- 战略目标
- 业务价值
- 愿景
- 范围

示例：

建设校园卡自助系统，提高充值、挂失、查询等业务的自助化比例，降低人工窗口压力。

用户需求

回答“用户想用系统完成什么任务”。

特点：

- 面向用户任务
- 语言不够精确
- 常混合功能、界面和约束

示例：

学生希望在网页或移动端自助完成校园卡充值，并查看充值记录。

系统级需求

回答“系统必须提供什么能力和约束”。

特点：

- 精确
- 可验证
- 可追踪
- 面向系统交互和响应

示例：

系统接收充值请求后，应校验校园卡状态和支付结果；支付成功后创建充值记录并更新账户余额。

2.4 需求类型

功能需求

说明系统做什么。

示例：

- 用户登录。
- 查询校园卡余额。
- 提交充值订单。
- 查询充值记录。
- 挂失校园卡。

非功能需求

说明系统达到什么质量或约束。

常见维度：

- 性能

- 安全
- 可用性
- 可靠性
- 可维护性
- 可移植性
- 易用性

性能维度：

- 速度
- 容量
- 吞吐量
- 负载
- 时效性或实时性

外部接口需求

说明系统与外部系统、硬件、用户界面之间的交互约束。

示例：

- 与支付平台通过 HTTPS 通信。
- 与校园卡中心系统同步余额。
- 前端通过 REST API 调用后端服务。

约束

说明系统建设必须遵守的限制。

示例：

- 使用学校统一身份认证。
- 数据库存储在校内数据中心。
- 后端采用 Java Spring Boot。

数据需求

说明系统需要管理的数据对象、数据字段、完整性规则和保留策略。

示例：

- 充值记录包含记录编号、校园卡号、金额、支付渠道、支付状态、创建时间。
- 充值金额必须大于 0。

2.5 AI 辅助需求获取

AI 适合：

- 从访谈记录中提取功能点。
- 整理用户故事。
- 生成需求文档初稿。
- 把自然语言改写成结构化条目。
- 检查编号、格式和冲突线索。

AI 的薄弱点：

- 容易遗漏隐含业务规则。
- 容易忽略系统边界。
- 容易生成不符合真实组织流程的功能。
- 对领域例外、权限规则、历史系统接口理解不足。

推荐流程：

1. Prompt：给出背景、角色、范围、输出格式。
 2. Generate：生成需求条目或模型草图。
 3. Review：人工核对业务规则、边界、异常和术语。
 4. Refine：补充约束、优先级、验收标准和追踪关系。
-

2.6 用例建模

用例描述系统与外部对象之间的一组交互行为。它从用户目标出发，描述系统如何向参与者提供有价值的服务。

Cockburn 的区分：

- 用例：包含一个用户目标下的所有场景。
- 场景：用例中的一条具体路径。

基本元素

- 参与者 Actor：系统外部与系统交互的角色。
- 用例 Use Case：系统提供的服务。
- 系统边界：区分系统内部和外部。
- 关系：关联、包含、扩展、泛化。

参与者可以是人、外部系统、硬件设备或定时器。参与者位于系统边界外，用例位于系统边界内。

include 与 extend

include：

- 基本用例每次执行时都要执行被包含用例。
- 方向：基本用例指向被包含用例。
- 用于抽取多个用例共享的公共行为。

extend：

- 扩展用例在特定条件下插入基本用例。
- 方向：扩展用例指向基本用例。
- 用于异常、可选、附加行为。

示例：

- “充值” include “身份验证”：每次充值都要验证用户身份。
- “充值失败处理” extend “充值”：只有支付失败或写入余额失败时触发。

用例粒度

合格用例通常满足：

- 对应一个业务事件。
- 由参与者发起。
- 在连续时间内完成。
- 对参与者有业务价值。

常见错误：

- 把增删改查拆成太细的技术操作。
- 把同一业务目标拆散。
- 把登录、数据校验、数据库连接当作独立业务用例。
- 在用例图里写实现细节。

Cockburn 层次

- 概要层：范围太大，适合说明系统目标。
- 用户目标层：最适合作为主用例。
- 子功能层：粒度较小，常用于 include 或内部步骤。

2.7 用例描述模板

十项模板：

1. 用例编号
2. 用例名称
3. 参与者
4. 触发条件
5. 前置条件
6. 后置条件

- 7. 基本流程
- 8. 扩展流程
- 9. 特殊需求
- 10. 备注

前置条件是用例开始前已经成立的系统状态，不是“用户输入账号密码”这类步骤。

示例：充值用例简表

项目	内容
用例编号	UC-Recharge
用例名称	校园卡充值
参与者	持卡人
触发条件	持卡人选择充值功能
前置条件	持卡人已通过身份认证，校园卡状态正常
后置条件	充值成功时账户余额增加，并生成充值记录
基本流程	选择充值金额；系统创建充值订单；调用支付接口；支付成功后更新余额；显示结果
扩展流程	支付失败时记录失败原因并提示用户；校园卡状态异常时拒绝充值
特殊需求	充值结果写入和记录生成保持一致

2.8 用户故事与 BDD

用户故事常用格式：

作为 < 角色 >，我想要 < 能力 >，以便 < 业务价值 >。

示例：

作为持卡人，我想要在线充值校园卡，以便在食堂和门禁场景继续使用校园卡。

BDD 使用 Given-When-Then：

Given 持卡人已登录且校园卡状态正常
When 持卡人提交 100 元充值并完成支付
Then 系统增加校园卡余额并生成一条成功充值记录

ATDD 强调先定义验收测试，再驱动设计和实现。

2.9 需求分析模型

常用模型：

- 用例图
- 概念类图
- 顺序图
- 状态图
- ER 图
- OCL 约束

面向对象需求分析输出：

- 用例图和用例描述
- 概念类图或领域模型
- 顺序图
- 状态图
- OCL 约束

2.10 概念类图

概念类图描述问题域中的概念及其关系，属于分析模型。

它与设计类图的区别：

项目	概念类图	设计类图
阶段	需求分析	详细设计
关注	领域概念	软件类、接口、方法
内容	类名、属性、关联、多重性	属性、方法、可见性、接口、继承、依赖

项目	概念类图	设计类图
来源	用例、业务文本、领域知识	架构、设计原则、实现技术

从文本提取概念类

步骤：

1. 收集用例描述、访谈记录和业务规则。
2. 标记名词和名词短语。
3. 合并同义词。
4. 删除纯属性、实现概念、外部系统、重复词和临时消息。
5. 与业务人员确认剩余概念。
6. 为概念添加属性、关联和多重性。

判断一个词能否成为概念类：

- 系统需要长期记录它。
- 它拥有多个属性。
- 它参与业务规则。
- 它与其他概念存在稳定关系。

示例：

- “充值记录”是概念类，因为系统要保存记录编号、金额、支付状态、时间等属性。
- “金额”通常是属性，不是概念类。
- “支付平台”是外部系统，在系统边界外。

关联与多重性

动词短语常提示关联：

- 用户拥有校园卡。
- 校园卡产生充值记录。
- 充值记录对应支付订单。

数量词提示多重性：

- 一个用户可以拥有多张校园卡。
- 一张校园卡可以有多条充值记录。
- 一条充值记录对应一个支付订单。

2.11 顺序图

顺序图描述对象之间按时间顺序发生的消息交互。

画法要点：

- 先选一个用例。
- 从基本流程开始。
- 每个交互步骤对应消息。
- 使用 `alt` 表示分支，使用 `opt` 表示可选步骤。
- 消息顺序必须与业务流程一致。

充值用例的典型对象：

- 持卡人
- 充值界面
- 充值控制器
- 充值服务
- 校园卡仓储

- 充值记录仓储
- 支付网关

典型流程：

1. 持卡人提交充值请求。
2. 控制器调用充值服务。
3. 服务查询校园卡状态。
4. 服务创建充值记录或充值订单。
5. 服务调用支付网关确认支付结果。
6. 服务更新余额。
7. 服务保存充值记录。
8. 控制器返回充值结果。

2.12 状态图

状态图描述一个对象在生命周期内的状态变化。

适合建模：

- 订单
- 校园卡
- 支付记录
- 设备
- 会话

充值记录状态示例：

- Created
- Paying
- Success

- Failed
- Cancelled

状态图可以直接指导测试：

- 每条合法迁移对应一个测试。
- 每条非法迁移对应一个异常测试。

2.13 OCL

OCL 是附着在 UML 模型上的声明式约束语言。

特点：

- 无副作用。
- 说明不变式、前置条件和后置条件。
- 可表达集合约束。

常见元素：

- `inv`：不变式。
- `pre`：前置条件。
- `post`：后置条件。
- `@pre`：操作执行前的值。
- `forall`、`exists`、`select`、`collect`：集合操作。

示例：

```
context Recharge
inv PositiveAmount:
    self.amount > 0

context CampusCard::recharge(amount)
pre ValidAmount:
    amount > 0
post BalanceIncreased:
    self.balance = self.balance@pre + amount
```

2.14 SRS 与需求验证

SRS 是系统级需求规格说明，从产品和系统视角列出需求。

用例文档和 SRS 的区别：

项目	用例文档	SRS
视角	用户交互	系统能力
组织方式	按用户目标	按功能、接口、数据、质量属性
重点	场景和流程	可验证、可追踪的需求条目

SRS 条目要使用刺激-响应结构：

当 < 刺激 > 发生时，系统应 < 响应 >。

示例：

当持卡人完成支付且支付平台返回成功结果时，系统应更新校园卡余额并生成成功充值记录。

2.15 好需求的标准

常见标准：

- 正确性
- 完整性
- 一致性
- 可读性
- 可修改性
- 可验证性
- 可追踪性

验证方法：

- 同行评审

- 原型验证
- 仿真
- 测试用例推导
- 追踪矩阵检查

2.16 需求追踪

前向追踪：

需求 → 设计 → 代码 → 测试

后向追踪：

代码或设计 → 需求来源

追踪矩阵常见列：

- 需求编号
- 需求来源
- 设计模块
- 代码模块
- 测试用例
- 状态

需求属性：

- 编号
- 描述
- 理由
- 来源
- 优先级

- 稳定性
- 状态
- 负责人
- 版本
- 变更历史

2.17 歧义与冲突

歧义类型：

- 词法歧义：同一词有多种含义。
- 句法歧义：句子结构导致多种解释。
- 语用歧义：上下文或业务背景不足导致理解不一致。

冲突类型：

- 直接矛盾：两个需求对同一条件给出相反响应。
- 资源冲突：时间、预算、算力、设备或人员无法同时满足。
- 优先级冲突：不同利益相关者对需求重要性排序不同。

处理流程：

1. 标记冲突需求编号。
2. 说明冲突类型。
3. 找到对应利益相关者。
4. 召开评审。
5. 记录决策和理由。
6. 更新 SRS、追踪矩阵和变更历史。

2.18 Spec 驱动开发

SDD 链路：

Spec → Design → Tasks → Code

好的 Spec 包含：

- 问题背景
- 用户故事
- 功能范围
- 验收标准
- 非功能约束
- 不在范围内的内容
- 待确认问题

在 AI 协作中，Spec 是人与 AI 之间的工程协议。Spec 越清晰，AI 输出越容易被审查、测试和接入项目。

对应考试模板

7.4 需求题答题模板

写业务需求

BR-01 建设校园卡自助系统，提高校园卡充值、查询、挂失等业务的自助办理比例，减少人工窗口处理压力。

写用户需求

UR-01 作为持卡人，我想在线充值校园卡，以便在余额不足时继续使用校园卡。

写系统级功能需求

FR-01 当持卡人提交充值请求时，系统应校验持卡人身份、校园卡状态和充值金额。

FR-02 当支付平台返回支付成功结果时，系统应更新校园卡余额并生成成功充值记录。

FR-03 当持卡人查询充值记录时，系统应返回该校园卡的充值记录列表。

写非功能需求

NFR-01 系统应通过 HTTPS 提供充值接口。

NFR-02 充值成功响应时间应不超过 3 秒，不包含第三方支付平台处理时间。

NFR-03 系统应记录每次充值请求、支付结果和余额更新结果。

7.5 概念类图题模板

校园卡系统概念类：

- User
- CampusCard
- RechargeRecord
- PaymentOrder
- LossReport
- Transaction

主要关系：

- 一个 User 拥有零张或多张 CampusCard。
- 一张 CampusCard 对应零条或多条 RechargeRecord。
- 一条 RechargeRecord 对应一个 PaymentOrder。
- 一张 CampusCard 对应零条或多条 LossReport。
- 一张 CampusCard 对应零条或多条 Transaction。

概念类示例：

User

userId
name
studentNo

CampusCard

cardNo
balance
status

RechargeRecord

recordId
amount
status
createdAt

PaymentOrder

orderId
channel
status
paidAt

7.6 顺序图题模板

充值基本流程:

```
Actor -> RechargePage: 输入金额并提交
RechargePage -> RechargeController: POST /api/recharges
RechargeController -> RechargeService: recharge(command)
RechargeService -> CampusCardRepository: findByCardNo(cardNo)
CampusCardRepository --> RechargeService: CampusCard
RechargeService -> PaymentGateway: pay(order)
PaymentGateway --> RechargeService: PaymentResult
RechargeService -> CampusCard: recharge(amount)
RechargeService -> CampusCardRepository: save(card)
RechargeService -> RechargeRecordRepository: save(record)
RechargeService --> RechargeController: RechargeResult
RechargeController --> RechargePage: JSON result
```

异常分支:

- 校园卡不存在: 抛出 CardNotFoundException。

- 校园卡已挂失：返回充值拒绝。
- 支付失败：保存失败记录并返回失败原因。
- 保存余额失败：执行事务回滚并记录错误。

7.7 状态图题模板

校园卡状态：

```
Normal -> Lost: reportLost
Lost -> Normal: cancelLossReport
Normal -> Frozen: freeze
Frozen -> Normal: unfreeze
Normal -> Cancelled: cancel
Lost -> Cancelled: cancel
Frozen -> Cancelled: cancel
```

充值记录状态：

```
Created -> Paying: startPayment
Paying -> Success: paymentSucceeded
Paying -> Failed: paymentFailed
Created -> Cancelled: cancel
Failed -> Created: retry
```

测试点：

- 正常迁移要测。
- 非法迁移要测。
- 每个状态的允许操作要测。

快速背诵清单

需求工程

- 活动：获取、分析、规格说明、验证、管理。

- 层次：业务需求、用户需求、系统级需求。
- 类型：功能、非功能、接口、约束、数据。
- 用例：用户目标、参与者、系统边界、include、extend。
- SRS：系统视角、刺激-响应、可验证、可追踪。